基座模型选型分析

一、选型结论

角色	推荐模型	理由
Teacher (27B级)	**Qwen3.5-27B**	原生多模态、同架构族蒸馏路径最直接
Student (4B级)	**Qwen3.5-4B (no-thinking)**	homeobjects 实测全面领先，空间关系理解大幅优于 Qwen3-VL-8B

一句话总结： Teacher 用 thinking 来"想清楚"（仅限离线单次推理），Student 用 no-thinking 来"做快速"。蒸馏是把"想清楚"的结论教给"做快速"的模型。推理链是 Teacher 的内部工具，不是 Student 的输出目标。

二、架构对比：Qwen3-VL vs Qwen3.5

维度	Qwen3-VL	Qwen3.5
架构范式	**Encoder-Decoder**（独立 ViT + MLP 投影 + LLM）	**Early Fusion**（视觉和文本 token 在统一 Transformer 中处理）
视觉处理	独立 ViT 编码器 + DeepStack 多层融合	原生多模态，无独立视觉编码器
模型类	`Qwen3VLForConditionalGeneration`	`Qwen3_5ForConditionalGeneration` / `Qwen3_5MoeForConditionalGeneration`
Thinking 模式	无显式 toggle	支持 `enable_thinking`
视觉预算	保守（max_pixels=1.6M, max_frames=32）	更大（max_pixels=128x32x32, max_frames=768）
演进方向	已停止演进（官方不再发布新版本）	后续长期主线方向

核心判断： Qwen3-VL 的 **Encoder-Decoder** 架构是**过渡方案**，Qwen3.5 的 **Early Fusion** 是**长期方向**。选择 Qwen3.5 系列可以避免后续被迫迁移架构的风险。

2.1 Qwen3.5-4B 的混合注意力架构

Qwen3.5-4B 采用了与 Qwen3 完全不同的混合注意力架构：

```
Qwen3-4B:   36 层 × 全 Softmax Attention（标准 Transformer）
Qwen3.5-4B: 32 层 × 混合架构：
  8 个重复单元 × (
    3 × (Gated DeltaNet → FFN)    ← 线性注意力层（24 层）
    1 × (Gated Attention → FFN)    ← 标准 Softmax 注意力层（8 层）
  )
```

架构参数	Qwen3-4B	Qwen3.5-4B
总层数	36	32
全注意力层数	36（100%）	8（25%）
线性注意力层数	0	24（75%）
Q-heads（全注意力）	32	16
KV-heads（全注意力）	8	4

75% 的层采用线性注意力（Gated DeltaNet），维护固定大小的状态矩阵（不随序列长度增长），使 KV cache 压力大幅低于全注意力架构。

三、Thinking 模式机制与影响

3.1 控制机制

`enable_thinking` 是一个 **Jinja2 模板变量**，不修改模型权重、不改变网络结构——只改变输入 prompt：

- **开启 thinking**：模板仅生成 `<|im_start|>assistant\n`，模型自主决定是否生成 `<think>` 标签
- **关闭 thinking**：模板预填充空 think 块 `<think>\n\n</think>\n\n`，模型"看到"思考已完成，直接输出结论

Qwen3.5 小模型（0.8B/2B）thinking 默认关闭，大模型（27B+）默认开启。两种模式的采样策略也不同：Thinking 使用高温度（1.0）鼓励推理多样性，No-thinking 使用低温度（0.7）确保输出确定

性。

3.2 实测数据：两种模式的代价与收益

以下数据来自 **homeobjects_mini_v1** 基准（352 个任务），同一模型（Qwen3.5-4B）、同一数据、仅切换 thinking 开关：

代价侧——Thinking 的额外开销：

维度	Thinking	No-Thinking	差异倍数
平均推理耗时	2.422s/task	0.529s/task	**4.6x**
总推理耗时	852.7s	186.1s	**4.6x**
总输出 token	5,315,540	1,801,190	**2.95x**

收益侧——两套 benchmark 给出了矛盾的结论：

在 homeobjects 实际任务上，No-Thinking **全面胜出：**

指标	Thinking	No-Thinking	变化	胜出
Detection F1	0.7734	**0.8164**	+5.6%	No-Thinking
Detection Recall	0.7111	**0.8278**	+16.4%	No-Thinking
Table 召回	0.6288	**0.7727**	+22.9%	No-Thinking
几何关系准确率	0.7021	**0.8298**	+18.2%	No-Thinking
房间分类准确率	0.0000	**0.7059**	—	No-Thinking

但在 MMBench 通用 benchmark 上，Thinking **大幅领先：**

指标	Thinking	No-Thinking	变化	胜出
总分	**81.79%**	73.20%	-8.59	Thinking
空间关系	**71.11%**	46.67%	-24.44	Thinking
逻辑推理	**72.88%**	57.63%	-15.25	Thinking

3.3 矛盾的根源：两种"空间关系"能力

同样是"空间关系"指标，homeobjects 和 MMBench 的结论完全相反（82.98% no-thinking 胜 vs 71.11% thinking 胜）。这不是数据矛盾，而是两套 benchmark 考查的是**两种不同的空间能力**：

维度	MMBench 空间关系	homeobjects 几何关系
任务形式	**"A 和 B 的空间关系是什么？请解释"**	**"A 在 B 的哪一侧？输出 `left_of` 或 `right_of`"**
能力本质	**描述性空间推理**——需要组织文字解释	**感知性空间判断**——需要直接从视觉特征输出
Thinking 效果	推理链帮助组织多步分析，**有益**	推理链引入犹豫和自相矛盾，**有害**
输出长度	长文本描述（50-200 tokens）	单词标签（1-3 tokens）

核心结论：Thinking 的价值取决于任务是"需要想清楚再说"还是"看到就直接判断"。 前者受益于推理链，后者被推理链干扰。

3.4 "过度推理"的具体伤害机制

Thinking 在感知任务上不是"没有帮助"，而是**主动造成伤害**。实测中观察到三类具体问题：

1. **推理犹豫导致漏检**：thinking 链中出现"不确定是否算桌子"等犹豫措辞，最终漏掉被遮挡的目标（Table 召回从 77.27% 降至 62.88%）
2. **推理链破坏输出格式**：thinking 文本溢出到结构化输出区域，或因 `</think>` 缺失导致 content 为空（房间分类从 70.59% 崩溃至 0.00%）
3. **推理结论与最终输出自相矛盾**：思考过程得出"应左转"，实际输出"右转"（VLN 导航实测中多次出现）

3.5 分场景推荐策略

综合代价与收益，Thinking 模式的选择应严格按任务性质区分：

任务性质	推荐模式	原因
感知密集型（检测、定位、枚举）	**No-Thinking**	过度推理干扰快速感知，格式不稳定
推理密集型（空间推理、搜索规划）	**Thinking（仅限 Teacher 离线）**	推理链提升复杂空间推理质量
结构化短输出（bbox JSON、区域排序）	**No-Thinking**	输出格式稳定，不被推理链干扰
闭环导航动作输出	**No-Thinking**	

		VLN 实测证明 thinking 导致过早停止、格式溢出、上下文污染
端侧实时部署	**锁定 No-Thinking**	延迟从 2.4s 降至 0.5s，满足实时性约束
Teacher 数据生成	**按任务切换**	检测数据用 no-thinking，推理数据用 thinking

四、实测数据对比

4.1 homeobjects 基准测试（352 个任务）

数据来源：`vlm_candidate_evaluation_for_detection.md`（2026-04-07）

指标	Qwen3-VL-4B	Qwen3-VL-8B	Qwen3.5-4B (Thinking)	**Qwen3.5-4B (NoThinking)**
Detection F1	0.7988	0.8000	0.7734	**0.8164**
Referring Acc@0.5	0.5745	0.6064	0.7021	**0.7021**
空间关系准确率	69.15%	76.60%	70.21%	**82.98%**
房间分类准确率	70.59%	73.53%	0.00%**（思考推理导致内容截断，content 字段为空导致结果异常）**	**70.59%**
推理延迟	**0.361s**	0.664s	2.422s	0.529s

关键发现：

- Qwen3.5-4B (no-thinking) 在空间关系准确率上全面碾压 Qwen3-VL-8B（82.98% vs 76.60%），4B 模型超越 8B 模型
- Referring grounding 能力提升显著（0.7021 vs 0.6064），说明 Early Fusion 对细粒度空间定位有结构性优势
- 推理延迟可控（0.529s），满足部署要求

4.2 MMBench-EN Dev 通用基准

数据来源：`06_model_compare/` 目录下各 eval report

指标	Qwen3-VL-4B	Qwen3-VL-8B		

			Qwen3.5-4B (Thinking)	Qwen3.5-4B (NoThinking)
总分	83.68%	**84.97%**	81.79%	73.20%
空间关系	48.89%	51.11%	**71.11%**	46.67%
场景识别	**98.08%**	96.15%	92.31%	90.38%
OCR	**94.87%**	87.18%	82.05%	76.92%

关键发现：

- 通用 benchmark 上 Qwen3-VL-8B 总分最高，但**空间关系能力** Qwen3.5-4B thinking 模式大幅领先（71.11% vs 51.11%）
- No-thinking 模式在通用 benchmark 上显著弱于 thinking 模式，但在实际任务（homeobjects）上反而更优，说明通用 benchmark 不能完全代表任务表现

> **PS：homeobjects 与 MMBench 对于空间关系的任务存在不同**

- **MMBench** 的空间关系题目是描述性推理——需要用文字解释空间关系，thinking 链帮助组织答案
- **homeobjects** 的几何关系是直接感知判断——只需输出 `left_of` 或 `right_of`，thinking 链中的多步推理反而引入犹豫和错误

4.3 VLN 基准测试（Habitat Instance ImageNav）

模型	成功率	成功 episodes / 总 episodes	平均SPL	平均步数	最大步数	停止阈值距离	平均错误动作/episode
qwen3.5-4b-no-thinking	**24.0%**	12 / 50	0.072	312.3	501	5.0m	**1.0**
qwen3-vl-4b	**24.0%**	12 / 50	0.123	296.7	500	5.0m	0
qwen3.5-2b	22.0%	11 / 50	0.051	253.4	500	5.0m	—
qwen3-vl-8b	18.0%	9 / 50	0.126	230.6	500	5.0m	0
qwen3.5-4b-	16.0%	8 / 50	0.137	59.9	500	5.0m	615 总计

thinking							

> 注意：配置参数来源于 `configs/tasks/instance_imagenav.yaml`，所有模型共用同一配置，停止成功阈值 `success_distance = 5.0m`。
> qwen3.5-4b-no-thinking 数据来源：`analysis_report_v2.md`（2026-04-10），修复图像输入 + 关闭 thinking 后的结果。

关键对比：Qwen3.5-4B thinking vs no-thinking

指标	Thinking	No-Thinking	变化
成功率	16.0%	**24.0%**	**+8pp（+50%）**
平均步数	59.9	**312.3**	充分探索 vs 过早停止
平均错误动作/episode	615 总计	**1.0**	解析错误几乎消除

关闭 thinking 后成功率从 16% 提升至 24%（+50%），与 Qwen3-VL-4B 持平。动作解析错误从 615 次降至平均每 episode 1.0 次，thinking 内容不再污染动作解析，模型在真正根据视觉信息决策。

Thinking 模式在 VLN 中的致命问题（thinking 实测暴露）：

1. **推理链溢出：** 97.6% 的步骤输出了 `<think>...</think>` 推理链，**推理文字经常溢出到动作输出区域**，导致 615 次解析错误。推理结论与最终动作自相矛盾（思考"左转"但输出"右转"）
2. **上下文窗口被推理链占据：** 每步推理链大量占用有限上下文，历史动作窗口被压缩，多步导航时早期观测信息被"挤出"上下文
3. **过早停止：** 平均步数仅 59.9（其他模型均超 200 步），13 个 episode 在 30 步内就停止失败。thinking 推理链使模型在探索初期就"自信"地判断已找到目标

No-Thinking 模式的当前问题（no-thinking 实测暴露）：

1. **虚假停止：** 4 个 episode 在 5-18 步内误输出"停止"，距目标 6-18m，到达判断存在 false positive
2. **超时比例高：** 19/50（38%）episode 跑满 501 步超时，探索策略仍需优化
3. **目标识别不均衡：** bed 全部成功（3/3），但 sofa/toilet 全部失败（0/6），视觉识别能力因目标类型差异显著

其他关键发现：

- qwen3.5-2b 虽然参数最小，但通过持续探索（平均 253 步），其成功率仍高于 8B 模型，说明持续探索比停止判断更重要
- "更大模型不一定更好"：Qwen3-VL-8B（18%）< Qwen3-VL-4B（24%），更强模型产生更复杂输出反而干扰动作遵从性

4.4 三类数据源的交叉验证

数据源	任务类型	Thinking 影响	结论
homeobjects 检测基准（352 任务）	单次感知	F1 下降 5.6%，几何关系下降 18.2%，房间分类崩溃	No-thinking 全面优于 thinking
MMBench-EN Dev	单次推理	总分提升 8.59%，空间关系提升 24.44%	Thinking 在描述性推理上有优势
Habitat VLN 导航（50 episodes）	多步闭环执行	关闭 thinking 后成功率从 16% 提升至 24%（+50%），解析错误从 615 次降至 ~1 次/episode	Thinking 在闭环导航中**灾难性有害**，关闭后与 Qwen3-VL-4B 持平

三类数据一致指向：**Thinking 的推理质量收益无法弥补其在执行稳定性、输出格式、延迟和上下文效率上的系统性损害。**

五、Token 预算与部署约束

5.1 Thinking 对 Token 预算的影响

输出格式	No-Thinking	Thinking	差异
标准输出（Teacher 训练）	100-150 tokens	400-600 tokens（含推理链）	3-4x
压缩输出（Student 部署）	20-40 tokens	不适用（Student 锁定 no-thinking）	—

Student token 预算（输入+输出）验证：

任务级别	No-Thinking 总计	Thinking 总计	结论
房间级导航	**450-660**	830-1230	Thinking 超标
家具级找物	**600-860**	970-1420	Thinking 超标
人物级找人	**650-920**	1020-1470	Thinking 超标
高频轻量层	**120-270**	300-640	Thinking 超标

结论： Thinking 模式的输出 token 使所有任务级别的总 token 超出预算 50-100%，完全不可行。高频轻量层（4-5 Hz）下 Thinking 的 TPS 需求高达 3200+，完全失控。

5.2 Thinking Token 对 KV Cache 的影响

由于 Qwen3.5-4B 混合注意力架构（75% GDN 线性层 + 25% 全注意力层），thinking token 的 KV cache 成本大幅低于纯全注意力架构：

- **全注意力层（8 层）**：thinking token 线性增长 KV cache
- **GDN 层（24 层）**：固定大小状态矩阵，不随 thinking 链长度增长

但 thinking token 仍然**等价消耗上下文窗口长度配额**，多步导航中推理链累积污染上下文的问题依然存在。

六、总结

1. **Qwen3.5 系列是正确的长期选择**：Early Fusion 架构是官方主线方向，Qwen3-VL 已停止演进
2. **实测数据已充分验证**：Qwen3.5-4B (no-thinking) 在 homeobjects 基准上全面领先，空间关系能力（82.98%）是导航任务的关键差异化指标
3. **同架构族蒸馏路径最优**：Qwen3.5-27B → Qwen3.5-4B 避免跨架构蒸馏的额外复杂度
4. **Student 部署锁定 no-thinking**：检测/导航任务中 thinking 模式有害（延迟高、F1 低、房间分类崩溃、VLN 灾难性失败）
5. **Teacher 按需使用 thinking**：仅限离线单次推理的数据生成场景，任何多步闭环执行场景必须关闭
6. **三类独立数据源交叉验证结论高度一致**：Thinking 的推理质量收益无法弥补执行稳定性的系统性损害